2026 경기도 공유학교 · 인공지능 기초 · 3회차

지도학습 2 및 모델 평가

"이 모델, 정말 잘하는 걸까? — 재는 법과 올리는 법"

— 성취기준 [인일 03-04] 인공지능 모델의 성능을 비교하고 개선할 수 있다

— 오늘: 왜 평가하는가 → 성능 지표 → 개선하기 → Colab 실습(모델 비교표·튜닝)

— 오늘의 결과물: 개선 전후 비교표 + "내가 이 모델을 고른 이유" 2~3문장

INTRO

학습목표와 오늘의 흐름

강의 45분 + Colab 실습 60분 + 정리 15분 — 결과물은 "숫자 근거가 있는 결정"

학습목표

- 훈련·테스트를 나누어 **공정하게 평가**하는 이유를 말한다
- 정확도·정밀도·재현율·F1·혼동행렬을 읽고 **정확도의 함정**을 예로 든다
- 과적합을 그래프로 확인하고 **교차검증·튜닝**으로 개선한다
- 성능표를 근거로 **개선 방향을 결정**하고 이유를 쓴다

시간	내용	형태
0:00	인사 · 2회차 복습 · 오늘의 목표	강의 5분
0:05	PART 1 왜 평가하는가 — 외운 모델 vs 이해한 모델	강의 15분
0:20	PART 2 성능 지표 — 정확도의 함정 · 혼동행렬 · 회귀 지표	강의 15분
0:35	PART 3 개선하기 — 교차검증 · 튜닝 · 스케일링	강의 10분
0:45	PART 4 Colab 실습 — 비교표 → 진단 → 튜닝 → 전후 비교	실습 60분
1:45	결과물 공유 · 퀴즈 · 정리 · 다음 시간 예고	15분

실습 환경

준비물: 구글 계정 + 크롬 브라우저. 실습 파일 03_지도학습2_모델평가_실습_colab.ipynb는 선생님이 나눠 줍니다. 데이터는 코드 안에서 자동으로 불러오므로 업로드할 파일이 없습니다.

INTRO

지난 시간 복습 — 지도학습 세 가지 알고리즘

정답(y)이 있는 데이터로 규칙을 배우는 것이 지도학습. 회귀는 숫자를, 분류는 종류를 맞힌다

알고리즘	원리 한 줄	어울리는 문제	사람이 정하는 값
kNN	가장 가까운 이웃 k명의 다수결	분류 (경계가 구불구불)	k (이웃 수)
결정트리	"○○이 크면?" 질문을 이어 가며 나누기	분류·회귀, 설명이 필요할 때	깊이 (max_depth)
선형회귀	점들 사이로 가장 잘 맞는 직선	회귀 (숫자 예측)	(거의 없음)
로지스틱 회귀	직선으로 "확률"을 만들어 분류	분류 (예/아니오)	C (단순함 정도)

2회차에 만든 모델은 "훈련 데이터에서 몇 % 맞혔나"만 봤습니다. **그 점수를 믿어도 될까요?** 오늘의 출발점입니다.

오늘 새로 만나는 말: 과적합 · 훈련/테스트 · 혼동행렬 · 정밀도/재현율 · 교차검증 · 하이퍼파라미터 · 스케일링

01

PART 1

왜 평가하는가

외운 학생과 이해한 학생을 구별하려면 새 문제를 내야 한다

이 파트에서

1 외운 모델 vs 이해한 모델

2 과소적합 · 적합 · 과적합

3 훈련 / 검증 / 테스트

4 과적합을 눈으로 — 트리 깊이 실험

PART 1 · 왜 평가하는가

문제집 답을 외운 학생은 새 문제를 못 푼다

모델도 훈련 데이터를 통째로 외울 수 있다 — 그러면 훈련 점수는 100%, 새 데이터는 엉망

외운 학생 = 과적합 모델

- 문제집(훈련 데이터)은 100점
- 숫자 하나만 바뀐 새 문제는 틀림
- 규칙이 아니라 **답 자체**를 기억
- 너무 복잡한 모델·너무 적은 데이터에서 잘 생김

이해한 학생 = 잘 배운 모델

- 문제집도 잘 풀고 새 문제도 잘 풂
- **원리(패턴)**를 기억
- 조금 틀리더라도 새 데이터에 강함
- 우리가 원하는 모델

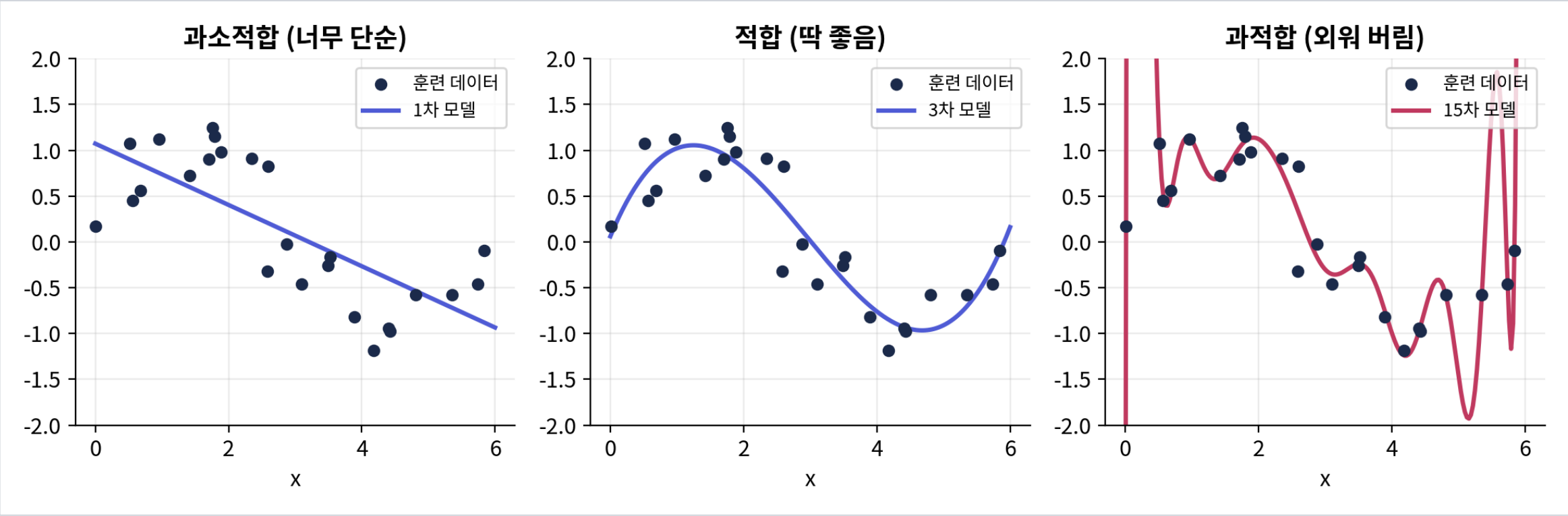
그래서 모델의 실력은 **본 적 없는 데이터**로 재야 합니다. 시험 문제를 미리 본 학생의 점수는 실력이 아니니까요.

1	2	3
훈련 데이터로 배우기	본 적 없는 데이터로 시험	점수 차이가 크면 "외운 것"

PART 1 · 왜 평가하는가

과소적합 · 적합 · 과적합 — 같은 점, 다른 곡선

너무 단순하면 훈련조차 못 맞히고(과소적합), 너무 복잡하면 훈련만 맞힌다(과적합)



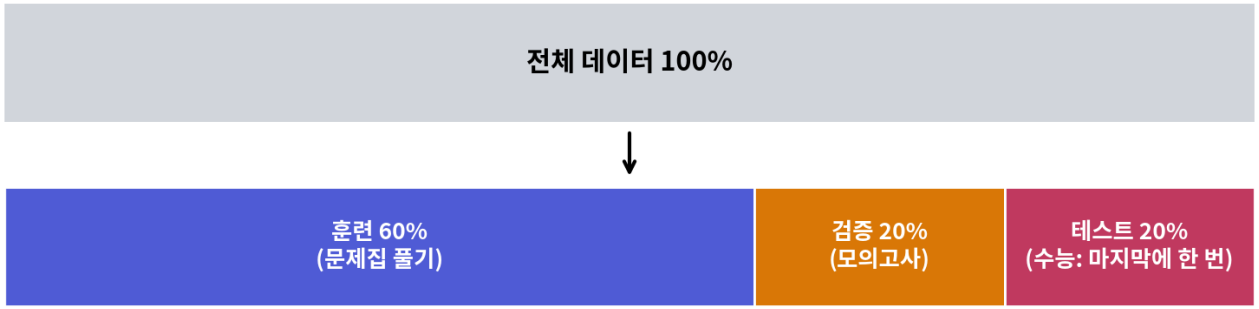
같은 25개 점을 1차·3차·15차 곡선으로 맞춘 결과. 15차 곡선은 모든 점을 지나가지만 새 점은 엉뚱하게 예측한다

상태	증상	처방
과소적합	훈련도 테스트도 낮음	더 복잡한 모델 · 특징 추가
과적합	훈련은 높은데 테스트가 낮음	단순하게(깊이 제한, k 늘리기) · 데이터 늘리기

PART 1 · 왜 평가하는가

데이터를 셋으로 나눈다 — 문제집 · 모의고사 · 수능

테스트 데이터는 "수능"처럼 마지막에 딱 한 번만 쓴다



훈련 규칙을 배우는 데이터 → fit()

검증 설정(k·깊이)을 고를 때 쓰는 데이터 → 교차검증이 대신

테스트 최종 실력 확인, **단 한 번** → score()

실습 3 **한 줄로 나누기**

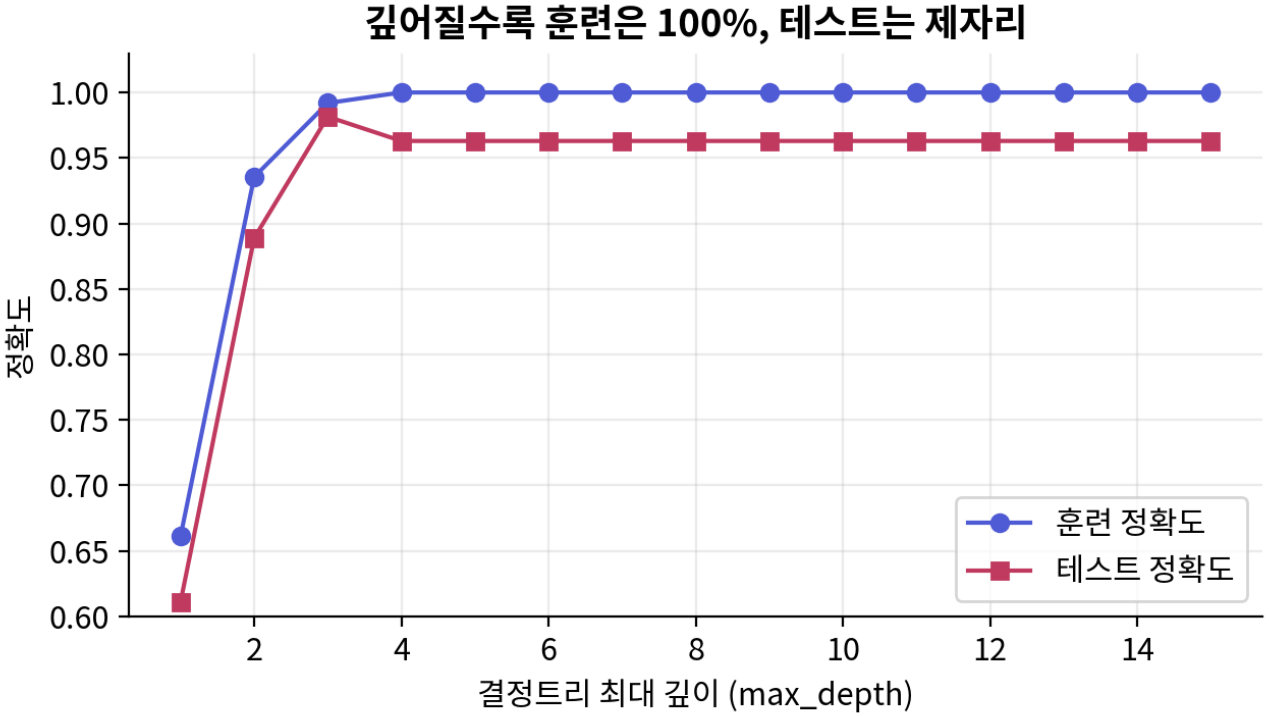
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
# test_size=0.3 → 30%를 테스트로   random_state=42 → 섞는 방법 고정   stratify=y → 품종 비율 유지
```

주의 테스트 점수를 보고 모델을 고치면, 그 순간 테스트는 "본 적 없는 데이터"가 아니게 됩니다. 설정은 검증(교차검증)으로, 테스트는 마지막에만!

PART 1 · 왜 평가하는가

과적합을 눈으로 — 트리를 깊게 팔수록 훈련만 오른다

와인 품종 데이터 · 결정트리 max_depth 1~15 (실습 7단계에서 직접 그린다)



그래프 읽기

- 깊이 1~2: 둘 다 낮음 → **과소적합**
- 깊이 3: 테스트 0.981로 **최고**
- 깊이 4+: 훈련 1.000, 테스트 0.963 제자리 → **과적합**
- 답: 테스트가 가장 높으면서 **가장 단순한** 깊이 3
- 점수가 같으면 단순한 쪽 (오컴의 면도날)

깊이·k처럼 **사람이 정하는 값**을 하이퍼파라미터라고 합니다. 좋은 값은 데이터마다 달라서 직접 시험해 봐야 합니다 (PART 3).

02

PART 2

성능 지표

"몇 % 맞혔나"만으로는 모자라다 — 문제의 목적에 맞는 자를 고르자

이 파트에서

1 정확도와 그 함정

2 혼동행렬

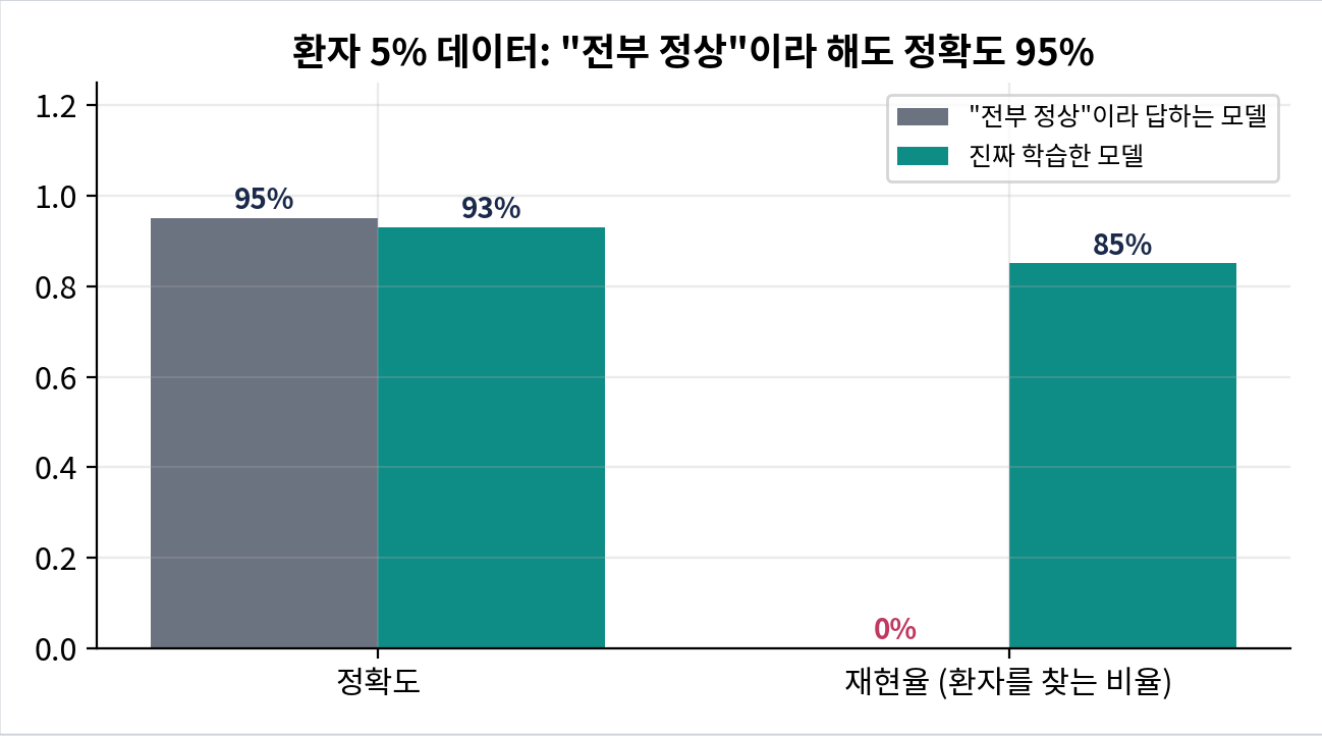
3 정밀도 · 재현율 · F1

4 회귀 지표 MAE · RMSE · R^2

PART 2 · 성능 지표

정확도 95%인데 쓸모없는 모델이 있다

환자가 5%뿐인 데이터: "전부 정상"이라고만 답해도 정확도 95%



정확도의 함정

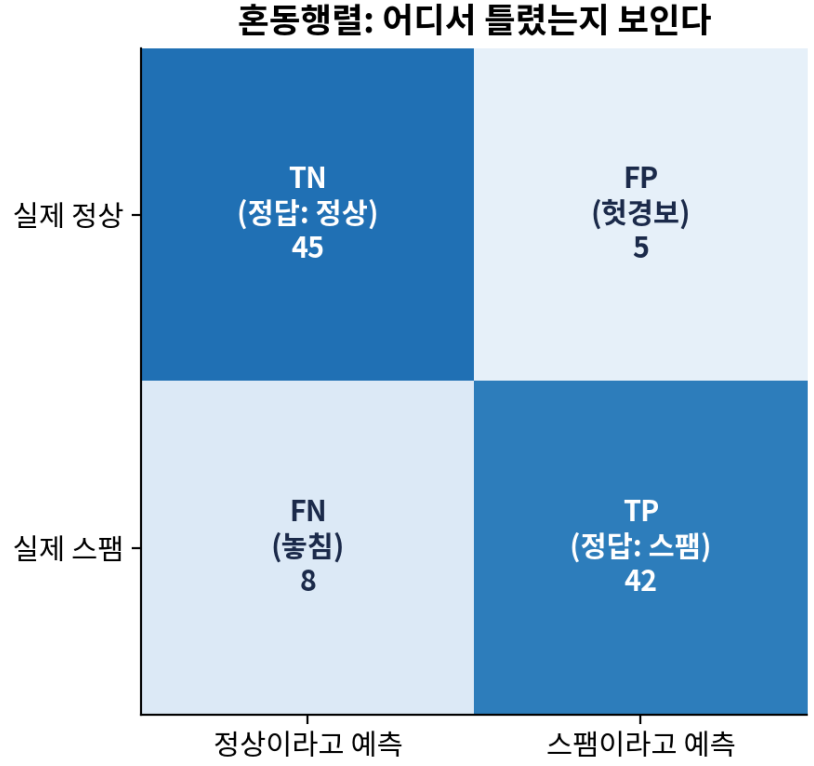
- 정확도 = 맞힌 개수 ÷ 전체
- 정답이 한쪽으로 **쏠린(불균형)** 데이터에서는 속는다
- "전부 정상" 모델: 정확도 95%, 환자 발견 **0명**
- 병 진단·사기 탐지·불량 검출이 모두 이런 데이터
- → 다른 자(지표)가 필요하다

실습 6단계에서 DummyClassifier(strategy="most_frequent")로 이 "전부 정상" 모델을 직접 만들어 비교합니다.

PART 2 · 성능 지표

혼동행렬 — 어디서, 어떻게 틀렸는지 보인다

행 = 실제, 열 = 예측. 대각선이 정답, 대각선 밖이 실수



칸	뜻 (스팸 필터 예)
TP	스팸을 스팸이라 함 (정답)
TN	정상을 정상이라 함 (정답)
FP	정상을 스팸이라 함 — **헛경보**
FN	스팸을 정상이라 함 — **놓침**

정확도 = (TP + TN) ÷ 전체 = (42 + 45) ÷ 100 = 0.87
실수의 종류(FP vs FN)에 따라 **피해가 다르다**는 것이 핵심

실습 5

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred)    # 행=실제, 열=예측
```

PART 2 · 성능 지표

정밀도와 재현율 — 헛경보가 싫은가, 놓침이 싫은가

두 지표는 시소: 한쪽을 올리면 다른 쪽이 내려간다 → 문제의 목적이 결정

정밀도 (Precision)

"스팸이라고 한 것 중
진짜 스팸은 몇 %?"

TP / (TP + FP)

헛경보(FP)를 줄이고 싶을 때

재현율 (Recall)

"진짜 스팸 중
찾아낸 건 몇 %?"

TP / (TP + FN)

놓침(FN)을 줄이고 싶을 때 (병 진단)

상황	중요한 지표
병 진단	재현율 (놓치면 큰일)
스팸 필터	정밀도 (정상 메일 잃으면 곤란)
화재 감지	재현율
유튜브 추천	정밀도 (엉뚱한 추천 싫음)

F1 점수 = 정밀도와 재현율의 조화 평균. 둘 다 중요하거나 데이터가 불균형할 때 정확도 대신 씁니다. classification_report() 한 줄이면 품종별 정밀도·재현율·F1이 다 나옵니다.

PART 2 · 성능 지표

회귀는 "얼마나 빗나갔나"를 잰다 — MAE · RMSE · R^2

숫자를 맞히는 문제에는 맞다/틀리다가 없다

세 가지 자

$$\text{MAE} = \text{평균}(\,|\text{실제} - \text{예측}|\,)$$

$$\text{RMSE} = \sqrt{\text{평균}(\,(\text{실제} - \text{예측})^2\,)}$$

$$R^2 = 1 - \text{모델 오차} \div \text{"평균으로 찍은" 오차}$$

지표	읽는 법
MAE	"평균 5.5점 틀린다" — 단위 그대로, 직관적
RMSE	큰 오차를 더 무겁게 벌줌 · 항상 MAE 이상
R^2	1 = 완벽, 0 = 평균으로 찍은 수준, 음수 = 평균보다 못함

공부시간·수면 → 성적 (실습 12단계)	MAE	RMSE	R^2
선형회귀	5.53점	7.46점	0.747
결정트리 회귀 (깊이 제한 없음)	7.65점	9.83점	0.561
결정트리 회귀 (깊이 3)	5.86점	7.60점	0.738

깊이 제한 없는 트리는 회귀에서도 **과적합** → 오차가 커진다. 분류든 회귀든 원리는 같다.

03

PART 3

개선하기

점수를 올리는 세 가지 도구 — 교차검증 · 하이퍼파라미터 튜닝 · 스케일링

이 파트에서

1 교차검증: 한 번 나누기의 운 없애기

2 하이퍼파라미터 튜닝: GridSearchCV

3 스케일링: kNN이 약했던 진짜 이유

4 개선 전 → 후

PART 3 · 개선하기

교차검증 — 한 번 나누기의 "운"을 없앤다

훈련 데이터를 5조각으로 나눠 5번 시험하고 평균 (테스트 데이터는 아직 안 쓴다)

데이터를 5조각으로 → 매번 다른 조각을 검증에

1회	검증	훈련	훈련	훈련	훈련	점수1
2회	훈련	검증	훈련	훈련	훈련	점수2
3회	훈련	훈련	검증	훈련	훈련	점수3
4회	훈련	훈련	훈련	검증	훈련	점수4
5회	훈련	훈련	훈련	훈련	검증	점수5

최종 점수 = 5개 점수의 평균 (한 번 나누기의 운을 없앤다)

왜 필요한가

- 어떤 데이터가 테스트에 들어가느냐에 따라 점수가 **오르내림**
- 5개 점수의 **평균**은 운이 덜 섞인 실력
- 5개 점수의 **편차(±)**는 안정성
- 설정(k·깊이)을 고를 때 쓰는 "모의고사"

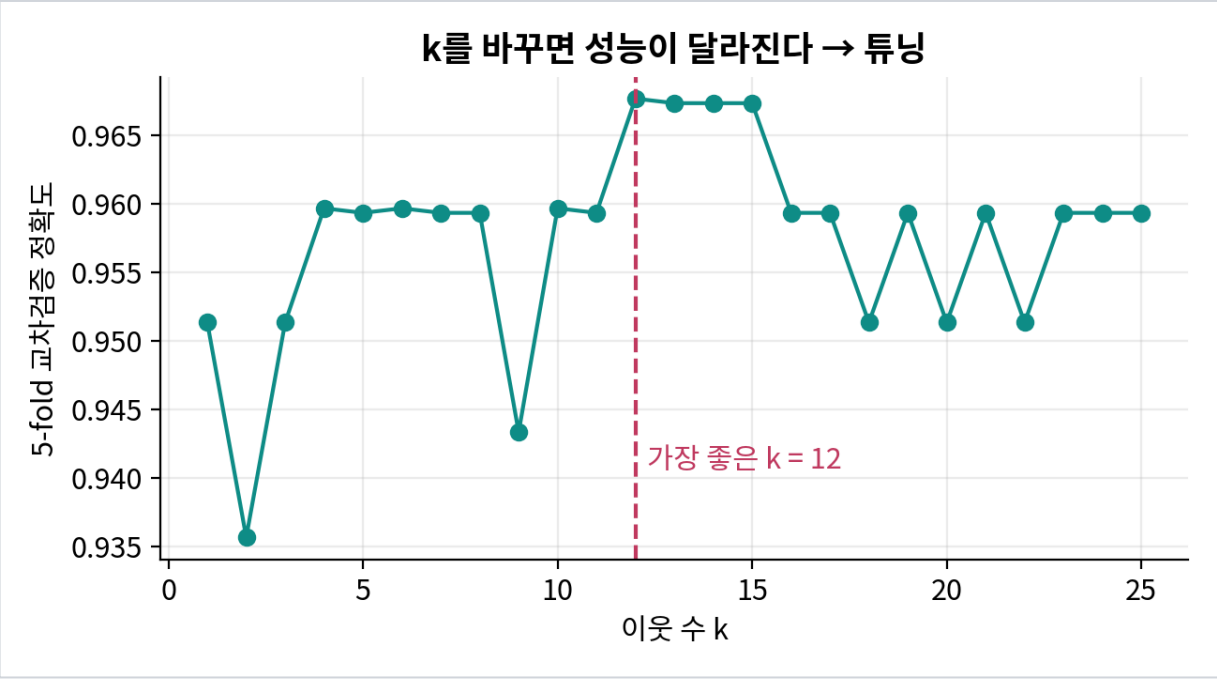
실습 9 한 줄이면 된다

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X_train, y_train, cv=5)     # 5번 시험
print(scores.round(3), scores.mean().round(3))              # 예: [0.92 0.96 1. 1. 0.917] 0.959
```

PART 3 · 개선하기

하이퍼파라미터 튜닝 — 후보를 전부 시험해 본다

GridSearchCV = 후보 값 × 교차검증을 자동으로 돌려 가장 좋은 설정을 고른다



kNN의 k를 1~25까지 바꾼 5-fold 교차검증 점수 — 이 데이터에서는 k=12가 최고

실습 10

```
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(
    make_pipeline(StandardScaler(), KNeighborsClassifier()),
    param_grid={'kneighborsclassifier__n_neighbors': range(1, 26)},
    cv=5)
grid.fit(X_train, y_train)      # 25개 후보 × 5번 = 125번 학습
grid.best_params_               # → k = 12
grid.best_estimator_            # 최적 설정으로 다시 학습한 모델
```

모델	하이퍼파라미터	작을수록	클수록
kNN	k (이웃 수)	복잡 (과적합 쪽)	단순 (과소적합 쪽)
결정트리	max_depth (깊이)	단순	복잡 (외움)
로지스틱 회귀	C	단순	복잡

PART 3 · 개선하기

kNN이 약했던 진짜 이유 — 열마다 숫자 크기가 다르다

스케일링(StandardScaler) 하나로 kNN 테스트 정확도 0.722 → 0.944

와인 데이터	alcohol	hue	proline
평균	13.0	0.96	747
최댓값	14.8	1.71	1680

kNN은 **거리**로 이웃을 찾는다. proline 차이가 수백이면 hue 차이 0.5는 있으나 마나 → 거리가 큰 열에 **독점**된다.

StandardScaler

- 모든 열을 **평균 0, 표준편차 1**로 맞춤
- make_pipeline(StandardScaler(), 모델)로 한 덩어리
- 훈련에서 배운 평균·표준편차를 테스트에 그대로 적용 (정보 유출 방지)
- 거리·기울기를 쓰는 kNN·로지스틱·선형회귀에 효과 / 결정트리는 불필요

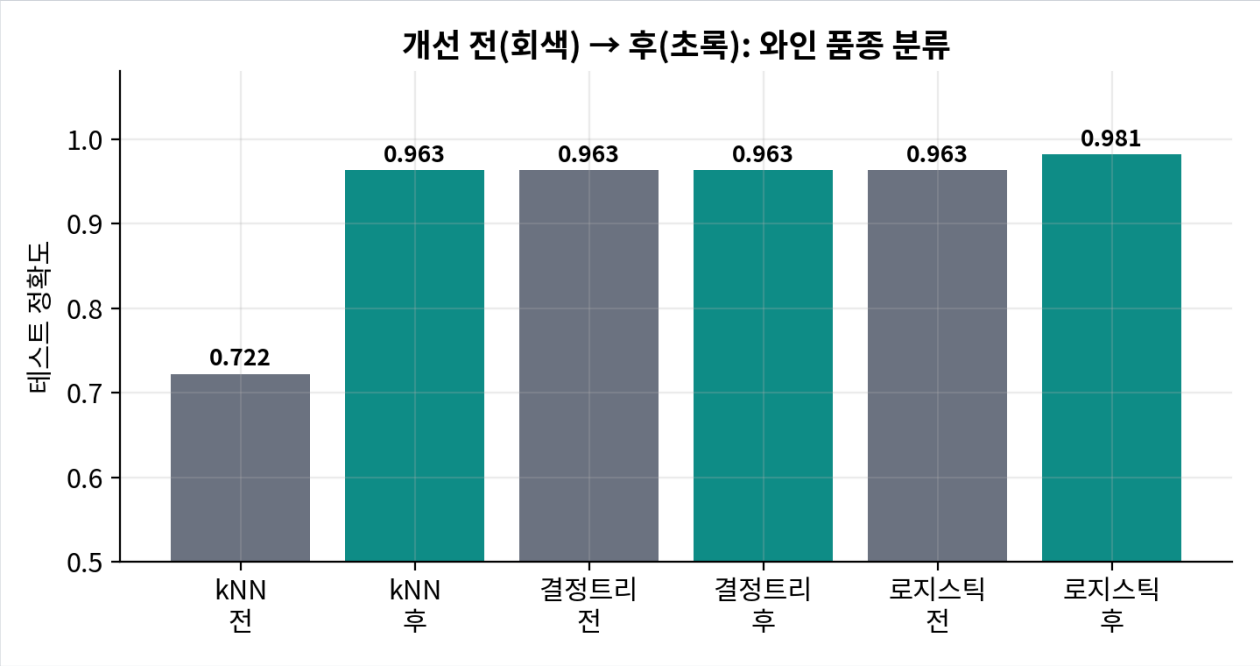
실습 8 **전처리만으로 22%p**

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
knn_scaled = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)).fit(X_train, y_train)
knn_scaled.score(X_test, y_test)     # 0.722 → 0.944  (모델은 그대로, 전처리만 바꿈)
```

PART 3 · 개선하기

개선 전 → 후 — 실습에서 만들 최종 비교표

와인 품종 분류 · 테스트 데이터 54병 · 실제 실행 결과



모델	전	후
kNN	0.722	**0.963**
결정트리	0.963	0.963 (깊이 3)
로지스틱 회귀	0.963	**0.981**

kNN: 스케일링 + k=12
결정트리: 깊이 3·잎 4 (점수 같음, 더 단순)
로지스틱: 스케일링

1 같은 조건으로 비교표

2 목적에 맞는 지표 읽기

3 과적합/과소적합 진단

4 튜닝 · 전처리

5 테스트로 최종 확인 → 근거 쓰기

04

PART 4

Colab 실습

모델 비교표 → 지표 읽기 → 과적합 실험 → 튜닝 → 개선 전후 비교 → 결정 근거 쓰기 (60분)

이 파트에서

1 노트북 열기 · 순서대로 실행

2 셀 1~6: 비교표 · 혼동행렬

3 셀 7~12: 진단 · 튜닝 · 회귀

4 직접 해보기 · 결과물 제출

PART 4 · Colab 실습

실습 시작 — Colab에 노트북 올리고 순서대로 실행

오류가 나면 "런타임 → 런타임 다시 시작" 후 처음부터

1	2	3	4
colab.research.google.com 접속 (구글 로그인)	파일 → 노트 업로드 → 03_..._colab.ipynb	셀 클릭 → Shift+Enter (위에서부터)	셀 1에서 "준비 완료!" 확인

실습 규칙

- 셀은 **위에서부터 순서대로** — 건너뛰면 변수가 없다는 오류
- 출력의 숫자를 **읽고 옆 사람과 말로 설명**해 보기
- 🧩 직접 해보기는 정답을 보기 전에 시도
- 막히면 HTML 수업자료의 "막혔을 때"를 먼저 확인

자주 나는 오류

- NameError: X_train is not defined → 위 셀을 안 돌림
- 그래프 한글이 □□ → 런타임 다시 시작 후 셀 1부터
- Invalid parameter → 옵션 이름 오타 (밑줄 2개 __)
- SyntaxError → 따옴표·괄호 짝 확인

데이터는 scikit-learn 내장 와인 데이터(178병·13성분·품종 3종). 인터넷 다운로드나 파일 업로드가 필요 없습니다.

PART 4 · Colab 실습

실습 단계표 — 셀마다 "무엇을 확인하는가"

셀 1~6 (25분): 나누기 · 비교표 · 혼동행렬 · 정확도의 함정

셀	내용	확인할 것
1	도구 불러오기 · 한글 폰트	"준비 완료!" 출력
2	와인 데이터 불러오기	(178, 13) · 품종별 59/71/48 → 거의 균형
3	훈련/테스트 나누기 (70:30, stratify)	훈련 124병 / 테스트 54병
4	kNN·결정트리·로지스틱 비교표	결정트리 훈련 1.000 vs 테스트 0.963 → 외운 흔적 / kNN 0.722 왜 낮을까?
5	혼동행렬 · classification_report	대각선 17·20·15 = 정답 / 품종0 1병을 품종1로 착각
6	"전부 정상" 더미 모델 vs 진짜 모델	정확도 0.957 vs 0.983 비슷 / 재현율 0.000 vs 0.615 하늘과 땅

PART 4 · Colab 실습

실습 단계표 — 진단하고 개선하기

셀 7~12 (25분): 과적합 실험 · 스케일링 · 교차검증 · GridSearchCV · 전후 비교 · 회귀 지표

셀	내용	확인할 것
7	트리 깊이 1~15 훈련/테스트 곡선	깊이 3에서 테스트 최고 0.981 / 4+는 훈련 1.000 제자리
8	StandardScaler + kNN	proline 평균 747 vs hue 0.96 / kNN 0.722 → 0.944
9	cross_val_score 5-fold	5개 점수가 조각마다 다름 / 로지스틱 평균 0.984 ±0.020 가장 안정
10	GridSearchCV (k 1~25 · 깊이·잎)	kNN 최적 k=12 / 트리 max_depth 3, min_samples_leaf 4
11	개선 전 → 후 비교표 (테스트 최종)	kNN 0.722→0.963 / 트리 0.963→0.963 / 로지스틱 0.963→0.981
12	회귀 지표 MAE·RMSE·R^2	선형회귀 MAE 5.53 · R^2 0.747 / 깊이 제한 없는 트리 과적합

PART 4 · Colab 실습

직접 해보기 2개 + 오늘의 결과물 제출

셀 13~15 (10분): 정답을 보기 전에 시도 → 제출 셀 출력 캡처

🧩 직접 해보기 1 — 로지스틱 회귀 C 튜닝

- 후보 [0.01, 0.1, 1, 10, 100]을 GridSearchCV로
- 옵션 이름: logisticregression__C
- 예상: 최적 C=1 · 교차검증 0.984 · 테스트 0.981

🧩 직접 해보기 2 — 판정 문턱값 바꾸기

- 환자 확률 문턱값 0.5 → 0.3 → 0.2 → 0.1
- 재현율 0.615 → **0.923**, 정밀도 1.0 → 0.800
- 병 진단이라면 어느 쪽이 나을지 토론

📝 **제출물 (셀 15)** 이름·학번 + 비교표 + 최종 선택 모델 + 이유 2~3문장 → 출력 화면 캡처
좋은 이유의 조건: ① 표의 숫자 인용 ② 교차검증 안정성(±) 언급 ③ 점수가 같으면 단순한 모델을 택한 이유 ④ 다음에 볼 지표 제안

시간이 남으면: load_breast_cancer()로 같은 과정 반복 → 이 문제에서 재현율이 왜 중요한지 써 보기

PART 5 · 정리와 평가

핵심 정리 — 오늘 가져갈 세 문장

결과물 공유 후 함께 읽기

1. 모델의 실력은 본 적 없는 데이터로 잰다. 훈련/테스트를 나누고, 설정은 교차검증으로 고르고, 테스트는 마지막에 한 번.

2. 정확도만 믿지 않는다. 불균형 데이터에서는 정밀도·재현율·F1, 회귀에서는 MAE·RMSE·R^2 — 문제의 목적이 지표를 정한다.

3. 개선은 순환이다. 같은 조건 비교표 → 과적합/과소적합 진단 → 튜닝·스케일링 → 최종 확인 → 숫자 근거로 결정.

1	2	3	4	5
비교표	지표 읽기	진단	튜닝·전처리	최종 확인 · 결정

PART 5 · 정리와 평가

퀴즈 — 세 문제

손들고 답하기 (정답은 다음 슬라이드)

Q1

- 훈련 정확도 100%, 테스트 정확도 70%.
- 이 모델의 상태는?

- ① 과소적합 ② 과적합
- ③ 데이터 과다 ④ 정상

Q2

- 환자 2% 데이터에서 "모두 정상"이라 답하는 모델의 정확도와, 꼭 봐야 할 지표는?

- ① 2%, 정확도 ② 98%, 재현율
- ③ 50%, MAE ④ 98%, R^2

Q3

- kNN 0.72 → StandardScaler 붙이니 0.94.
- 가장 알맞은 설명은?

- ① k가 바뀌었다
- ② 열마다 크기가 달라 거리가 큰 열에 좌우되던 것이 고쳐졌다
- ③ 테스트 데이터가 바뀌었다

퀴즈 정답과 해설

문제	정답	해설
Q1	② 과적합	훈련은 잘 맞히는데 새 데이터에 약함 = 외운 것. 깊이 제한·k 늘리기·데이터 늘리기로 완화
Q2	② 98%, 재현율	98%가 정상이므로 정확도 98%. 하지만 환자를 한 명도 못 찾음 → 재현율(진짜 환자 중 찾아낸 비율) 0
Q3	②	kNN은 거리를 쓰므로 값이 큰 열(proline)이 거리를 독점. 스케일링으로 모든 열을 같은 크기로

HTML 수업자료에 퀴즈 5문항이 더 있습니다 (보기를 누르면 해설이 나옵니다). 집에서 한 번 더 풀어 보세요.

PART 5 · 정리와 평가

실습 결과 평가 기준 — 무엇을 보는가

"모델 성능 결과를 바탕으로 개선 방향 결정" · 제출 캡처 1장으로 평가

항목	우수 (3)	보통 (2)	미흡 (1)
① 비교표 완성	3개 이상 모델의 전/후 점수	일부 모델만	표 없음
② 지표 해석	정확도 외 정밀도/재현율 또는 교차검증 편차 언급	정확도만 언급	해석 없음
③ 개선 방향 결정	표의 숫자를 인용해 최종 모델과 이유 2~3문장	이유 한 문장·근거 부족	선택만, 이유 없음
④ 직접 해보기	2개 모두 실행·결과 설명	1개 실행	미실행

핵심은 ③ — "점수가 가장 높아서"만이 아니라 **왜 그 숫자를 믿을 수 있는지**(안정성·단순함·목적에 맞는 지표)까지 쓰는 것

PART 5 · 정리와 평가

다음 시간 예고 — 4회차 비지도학습

정답이 없는 데이터를 컴퓨터가 스스로 묶는다

4회차에서 배울 것

- 정답(y) **없이** 배우는 비지도학습
- **군집(k-means)**: 비슷한 것끼리 묶기
- **차원축소(PCA)**: 13개 열을 2개로 줄여 그리기
- 오늘 쓴 와인 데이터를 정답 없이 다시 만난다

더 해보기 (선택)

- load_breast_cancer()로 오늘 과정 반복 — 재현율이 중요한 이유 쓰기
- RandomForestClassifier를 비교표에 추가
- 회귀에 KNeighborsRegressor 추가, 스케일링 전후 MAE 비교

오늘의 한 문장: **"점수는 재는 법에 따라 달라진다 — 그러니 재는 법부터 고른다."**